

Kotlin 中覆盖、接口与枚举类教学

Kotlin第一章第四节教学讲义（覆盖与接口、枚举类）

各位同学，今天咱们聚焦Kotlin中**“覆盖”机制和“接口、枚举类”**这三类核心知识点。覆盖是实现“多态”的关键，接口是解耦的利器，枚举类让常量管理更优雅。全程用“生活案例+代码实操”的方式，保证零基础也能轻松吃透！

一、覆盖方法：子类对父类方法的“个性化改造”

想象一个场景：“动物”都会“叫”，但“狗叫”和“猫叫”完全不同——这就是**覆盖 (Override)**：子类重写父类的方法，实现自己的逻辑。

1. 覆盖的核心规则

- 父类方法必须用 `open` 修饰（允许被重写）；
- 子类重写时必须用 `override` 修饰（明确声明重写）；
- 重写方法的签名（名称、参数、返回值）必须和父类一致；
- 可通过 `super.父类方法()` 调用父类的原始实现。

2. 实战案例：动物叫的个性化实现

Code block

```
1 // 父类：动物（方法加open，允许被重写）
2 open class Animal {
3     open fun makeSound() {
4         println("动物发出叫声")
5     }
6 }
7
```

```
8 // 子类：狗 (override重写父类方法)
9 class Dog : Animal() {
10     override fun makeSound() {
11         super.makeSound() // 可选：调用父类原始逻辑
12         println("汪汪汪! ")
13     }
14 }
15
16 // 子类：猫 (override重写父类方法)
17 class Cat : Animal() {
18     override fun makeSound() {
19         println("喵喵喵! ")
20     }
21 }
22
23 fun main() {
24     val dog = Dog()
25     dog.makeSound()
26     // 输出：
27     // 动物发出叫声
28     // 汪汪汪!
29
30     val cat = Cat()
31     cat.makeSound() // 输出：喵喵喵!
32 }
```

二、覆盖属性：子类对父类属性的“重新定义”

和方法一样，属性也可以被覆盖——子类重写父类的属性，定义自己的取值逻辑或赋值逻辑。

1. 覆盖属性的两种场景

场景	父类属性修饰符	子类实现要求
覆盖只读属性	<code>open val</code>	子类用 <code>override val</code> 重写，提供新值或计算逻辑
覆盖可写属性	<code>open var</code>	子类用 <code>override var</code> 重写，可修改取值/赋值逻辑

2. 实战案例：员工薪资的覆盖

Code block

```
1  // 父类：员工
2  open class Employee {
3      open val salary: Double = 5000.0 // 父类基础薪资 (open修饰, 允许被重写)
4  }
5
6  // 子类：高级员工 (覆盖薪资, 提供更高的基础值)
7  class SeniorEmployee : Employee() {
8      override val salary: Double = 8000.0
9  }
10
11 // 子类：销售员工 (覆盖薪资, 按业绩计算)
12 class SalesEmployee : Employee() {
13     private var performance: Double = 0.0
14     override var salary: Double
15         get() = 5000.0 + performance * 0.1 // 基础薪资+业绩10%提成
16         set(value) {
17             performance = value - 5000.0 // 反向计算业绩 (演示用)
18         }
19 }
20
21 fun main() {
22     val senior = SeniorEmployee()
23     println("高级员工薪资: ${senior.salary}") // 输出: 8000.0
24
25     val sales = SalesEmployee()
26     sales.salary = 7000.0 // 设置薪资→反向计算业绩
27     println("销售员工薪资: ${sales.salary}, 业绩: ${sales.performance}")
28     // 输出: 销售员工薪资: 7000.0, 业绩: 20000.0
29 }
```

三、接口：“行为标准”的协议

生活中“USB接口”是一个标准——不管是U盘、鼠标还是键盘，只要符合这个标准就能插在电脑上用。Kotlin中的接口 (**interface**) 也是如此：它规定了“类应该有哪些方法/属性”，但不关心具体实

现，实现接口的类必须“遵守协议”。

1. 接口的基本定义与实现

Code block

```
1  // 定义接口：充电协议（规定必须有充电方法）
2  interface Chargeable {
3      fun charge() // 抽象方法：必须由实现类实现
4      val voltage: Int // 抽象属性：必须由实现类提供值
5  }
6
7  // 实现接口：手机
8  class Phone : Chargeable {
9      override val voltage: Int = 220 // 实现接口属性
10     override fun charge() { // 实现接口方法
11         println("手机用Type-C接口充电，电压${voltage}V")
12     }
13 }
14
15 // 实现接口：电动车
16 class ElectricCar : Chargeable {
17     override val voltage: Int = 380
18     override fun charge() {
19         println("电动车用直流桩充电，电压${voltage}V")
20     }
21 }
22
23 fun main() {
24     val phone = Phone()
25     phone.charge() // 输出：手机用Type-C接口充电，电压220V
26
27     val car = ElectricCar()
28     car.charge() // 输出：电动车用直流桩充电，电压380V
29 }
```

2. 接口的“默认方法”（Kotlin特色）

接口中可以定义带实现的默认方法，实现类可以直接使用或重写。

Code block

```
1  interface Payable {
2      fun pay() // 抽象方法：必须实现
3      // 默认方法：带实现，实现类可直接用
4      fun showPayInfo() {
5          println("正在支付中...")
6      }
7  }
8
9  class WeChatPay : Payable {
10     override fun pay() {
11         println("微信支付扣款成功")
12     }
13     // 可选：重写默认方法
14     override fun showPayInfo() {
15         super.showPayInfo()
16         println("微信支付订单已生成")
17     }
18 }
19
20 fun main() {
21     val weChat = WeChatPay()
22     weChat.pay()
23     weChat.showPayInfo()
24     // 输出：
25     // 微信支付扣款成功
26     // 正在支付中...
27     // 微信支付订单已生成
28 }
```

四、接口属性：规定“必须有”的属性

接口不仅能定义方法，还能定义属性，但接口的属性**不能直接赋值**（因为接口没有“存储能力”），必须由实现类提供具体值或实现getter。

Code block

```
1  // 接口：商品协议
2  interface Goods {
```

```

3      val name: String // 抽象属性：必须由实现类赋值
4      val price: Double // 抽象属性：必须由实现类赋值
5      // 带默认getter的属性：非抽象，实现类可重写
6      val discountPrice: Double
7          get() = price * 0.8 // 默认8折
8  }
9
10 // 实现接口：手机商品
11 class PhoneGoods(
12     override val name: String,
13     override val price: Double
14 ) : Goods {
15     // 重写接口的discountPrice（比如会员7折）
16     override val discountPrice: Double
17         get() = price * 0.7
18 }
19
20 fun main() {
21     val phone = PhoneGoods("小米手机", 3999.0)
22     println("商品: ${phone.name}, 原价: ${phone.price}, 折扣价:
23         ${phone.discountPrice}")
24     // 输出: 商品: 小米手机, 原价: 3999.0, 折扣价: 2799.3
25 }

```

五、枚举类：“固定常量”的优雅管理

生活中“星期几”“性别”都是**固定的常量集合**，用枚举类（`enum class`）管理这类常量，比用普通常量更安全、更易读。

1. 枚举类的基本定义与使用

Code block

```

1 // 定义枚举类：星期
2 enum class WeekDay {
3     MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
4 }
5
6 fun main() {
7     // 获取所有枚举值

```

```

8     val allDays = WeekDay.values()
9     println("所有星期: ${allDays.contentToString()}")
10    // 输出: [MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY]
11
12    // 根据名称获取枚举值
13    val monday = WeekDay.valueOf("MONDAY")
14    println("指定星期: $monday") // 输出: MONDAY
15
16    // 枚举值的比较 (直接用==)
17    println("是否是周一: ${monday == WeekDay.MONDAY}") // 输出: true
18 }

```

2. 带属性和方法的枚举类（进阶）

枚举类可以包含**属性**和**方法**，让每个枚举值携带更多信息。

Code block

```

1  // 定义枚举类: 性别 (带中文描述属性)
2  enum class Gender(val desc: String) {
3      MALE("男性") {
4          override fun greet() = println("先生您好")
5      },
6      FEMALE("女性") {
7          override fun greet() = println("女士您好")
8      };
9
10     // 抽象方法: 每个枚举值必须实现
11     abstract fun greet()
12 }
13
14 fun main() {
15     val male = Gender.MALE
16     println("性别: ${male.desc}") // 输出: 男性
17     male.greet() // 输出: 先生您好
18
19     val female = Gender.FEMALE
20     println("性别: ${female.desc}") // 输出: 女性
21     female.greet() // 输出: 女士您好
22 }

```

总结与作业

核心知识点

- **覆盖方法/属性**：用 `override` 重写父类的 `open` 方法/属性，实现子类个性化逻辑；
- **接口**：用 `interface` 定义行为标准，实现类必须遵守协议，支持默认方法和属性；
- **枚举类**：用 `enum class` 管理固定常量集合，支持属性、方法和多态实现。

课后作业

1. 定义接口 `Flyable`，包含抽象方法 `fly()` 和默认方法 `showFlyHeight()`（打印“飞行高度：100米”）；
2. 定义两个类 `Bird` 和 `Plane` 实现 `Flyable` 接口，`Bird` 的 `fly()` 输出“鸟儿扇动翅膀飞行”，`Plane` 的 `fly()` 输出“飞机靠引擎飞行”；
3. 定义枚举类 `FlyType`，包含 `BIRD`（描述“生物飞行”）和 `MACHINE`（描述“机械飞行”），给每个枚举值添加 `flyDesc` 属性；
4. 在 `main` 函数中创建 `Bird` 和 `Plane` 对象，调用它们的 `fly()` 和 `showFlyHeight()` 方法，同时用枚举类描述它们的飞行类型。

试着自己写代码实现，有问题随时问哦！

（注：文档部分内容可能由 AI 生成）